SINGAPORE UNIVERSITY OF TECHNOLOGY AND DESIGN

50.039 Theory and Practice of Deep Learning

Name	ID
Sharon Ashok	1007915
Anbarasan Srivarthini	1008459
Jafira Binte Abdul Nassar	1007916

1 Executive Summary **5**
2 Dataset Handling & Performance Metrics **6**
2.1 Dataset 6
2.1.1 Dataset Overview 6
2.1.2 Use of Training vs Test Data 6
2.1.3 Target Variable 6
2.2 Preprocessing 7
2.2.2 Exploratory Data Analysis 7
2.2.3 Cleaning the Dataset 7
2.2.4 Splitting the Dataset 7
2.2.5 Feature Engineering 8
2.2.6 Encoding 8
2.2.7 Numerical Feature Scaling 9
2.3 Final Performance Metrics 9
2.3.1 PR-AUC 9
2.3.2 F2 Score 9
3 Deep Learning Models **10**
3.1 Model 1 - Naive Model 10
3.1.1 Overview 10
3.1.2 Model Architecture 10
3.1.3 Hyperparameters and Training Setup 10
3.1.4 Training Curves 11
3.1.5 Results 12
3.1.6 Analysis of Results 13
3.1.7 Conclusion 13
3.2 Model 2 14
3.2.1 Overview 14
3.2.2 Model Architecture 14
3.2.2 Improvements over Model 1 14
3.2.4 Hyperparameters and Training Setup 15
3.2.5 Training Curves 15
3.2.6 Results 17
3.2.7 Analysis of Results 17
3.2.8 Conclusion 18
3.3 Model 3 19
3.3.1 Overview 19
3.3.2 Model Architecture 19
3.3.3 Improvements over Model 2 19
3.3.4 Hyperparameters and Training Setup 20
3.3.5 Training Curves 20

3.3.6 Results 22
3.3.7 Analysis of Results 22
3.3.8 Conclusion 23
3.4 Final model 24
3.4.1 Overview 24
3.4.2 Pre-Processing 24
3.4.3 Model Architecture 24
3.4.4 Improvements over Model 3 25
3.4.5 Hyperparameters and Training Setup 26
3.4.6 Training Curves 28
3.4.7 Results 29
3.4.8 Analysis of Results 30
3.4.9 Conclusion 31
4 Machine Learning Baseline Models 32
4.1 Overview 32
4.2 Logistic Regression 32
4.2.1 Model Architecture 32
4.2.2 Hyperparameters 32
4.2.3 Training Curves 33
4.2.4 Results 34
4.2.5 Analysis of Results 34
4.2.6 Conclusion 35
4.3 Random Forest 35
4.3.1 Model Architecture 35
4.3.2 Hyperparameters 36
4.3.3 Training Curves 36
4.3.4 Results 37
4.3.5 Analysis of Results 38
4.3.6 Conclusion 38
4.4 XGBoost 39
4.4.1 Model Architecture 39
4.4.2 Hyperparameters 39
4.4.3 Training Curves 39
4.4.4 Results 41
4.4.5 Analysis of Results 42
4.4.6 Conclusion 42
5 Adversarial Attack 43
5.1 White-Box Attack 43
5.2 Vulnerabilities in our Model 43
Instructions for running the Model 45
Final Takeaways 46

Appendix and References **47**

1 Executive Summary

Fraudulent transactions pose a significant and persistent challenge within the financial and banking industry, leading to substantial monetary losses and undermining trust in financial systems. The ability to accurately and efficiently detect fraudulent transactions is therefore critical to operational integrity, risk management, and regulatory compliance of financial institutions.

In this project, we endeavoured to develop a deep learning-based model capable of identifying fraudulent transactions from complex financial data. Our approach aims to capture subtle patterns and anomalies that traditional rule-based or statistical methods may fail to detect.

By training the model on historical transaction data, we seek to enhance its ability to distinguish between legitimate and fraudulent behaviour, even in the presence of severe class imbalance and evolving fraud strategies.

The link to our project GitHub is found here:
https://github.com/Sri0666/IEEE-CIS-Fraud-Detection-Project

The following are the main portions of the report:

- Section 2: Dataset Handling & Performance Metrics
- Section 3.4: Final Model
- Section 4: Machine Learning Baseline models (to compare to state-of-the-art performance)
- Section 5: Attack + Model Vulnerabilities
- Final Takeways

Read the following section to know how to play with our model:

- Instructions for running the model

2 Dataset Handling & Performance Metrics

2.1 Dataset

The selected dataset for this project is the IEEE-CIS Fraud Detection Dataset, sourced from a Kaggle competition organised by the IEEE Computational Intelligence Society (IEEE-CIS). The primary objective of this dataset is to support the development of machine learning models capable of detecting fraudulent online transactions.

2.1.1 Dataset Overview

The dataset consists of anonymised real-world e-commerce transaction data provided by Vesta Corporation. It is designed to reflect the challenges of real fraud detection systems, including high dimensionality, missing values, and severe class imbalance.

The data is split into two main components:

1. Transaction Table - contains transaction-level information such as transaction amount, product category, and engineered features.
2. Identity Table - contains additional identity-related attributes such as device type, browser information, and anonymised user identifiers.

These tables are linked using a common key (TransactionID) and can be merged to form a unified dataset for modelling.

2.1.2 Use of Training vs Test Data

Although the dataset provides both training and test splits, only the training data was used in this study. The test dataset provided in the competition does not include ground truth labels (isFraud), as it is intended for submission and evaluation within the Kaggle competition framework.

Since the objective of this project is to build and evaluate supervised models independently rather than submit predictions to Kaggle, the test dataset was not utilised. Instead, the original training data was further split into training, validation, and test sets, enabling proper model development, hyperparameter tuning, and unbiased evaluation.

2.1.3 Target Variable

The target variable is defined to be **isFraud**, a binary class label indicating whether a transaction is fraudulent. 1 indicates a fraudulent transaction, and 0 indicates a legitimate transaction.

2.2 Preprocessing

Pre-processing was done to convert the raw IEEE-CIS fraud detection data into a consistent, model-ready format.

2.2.2 Exploratory Data Analysis

The exploratory data analysis examined the dataset structure, class distribution, and percentage of missing data. The merged dataset contains 434 columns, combining transaction and identity features, resulting in a high-dimensional feature space. The target variable isFraud is highly imbalanced, with fraudulent transactions accounting for 3.499% of the data. Missing value analysis showed that 414 columns contain missing data, with several identity-related features (e.g. id_24) having over 99% missing data.

2.2.3 Cleaning the Dataset

Based on the EDA findings, data cleaning was performed to handle missing values, reduce sparsity, and improve data quality. Columns with more than 75% missing values (208 columns) were removed, as they are too sparse to provide reliable information.

For the remaining features, missing values in numerical columns were imputed using the median, as it is robust to outliers and suitable for skewed financial data. Missing values in categorical columns were replaced with the label "missing", preserving missingness as a potential signal.

To reduce the impact of extreme values, numerical features were winsorised at the 1st and 99th percentiles, as neural networks are sensitive to large outliers. The identifier column TransactionID was also removed since it does not carry predictive information.

After cleaning the dataset, its shape was (590540, 225).

2.2.4 Splitting the Dataset

The cleaned dataset was split into training (80%), validation (5%), and test (15%) sets using a two-step process. Stratified sampling was applied based on the target variable, isFraud, to preserve the class distribution across all splits. Indices were reset after splitting to ensure clean and consistent data handling in subsequent steps.

2.2.5 Feature Engineering

The table below showcases the additional features added and their reasons.

Type of Features	Features	Reason
Time-based	hour of day, day-of-week proxy, weekend flag	To capture temporal fraud patterns.
Frequency	count encoding of card and email domain values (fitted on train set)	To represent how common or rare each value is in the training data.
Aggregation	per-card transaction amount mean/std/count, plus deviation and ratio to card mean	To capture cardholder spending behaviour and detect anomalies.
Transaction amount	log transform, cents (decimal part)	To normalise the distribution of transaction amounts and extract pricing patterns
Interaction	frequency encoding of card+address and card+email combinations	To capture cross-feature behaviour.
M-column summary	count of T / F / null across M1–M9	To gain a single summary of the features.

After feature engineering, all splits expanded from 208 to 272 columns, and the final feature space consisted of 19 categorical and 252 numerical variables.

2.2.6 Encoding

Categorical features were converted into integer representations using label encoding so they could be used as inputs to the neural network. For each categorical column, the encoder was fitted on the training set only, and the resulting category-to-integer mapping was then applied to the validation and test sets to prevent data leakage. Categories that appeared in the validation or test sets but were not seen during training were assigned to a dedicated out-of-vocabulary (OOV) index, allowing unseen values to be handled safely at inference time.

In total, 41 categorical columns were label encoded. The resulting vocabulary sizes varied substantially across features, from very low-cardinality variables such as ProductCD, card4, and card6 to high-cardinality variables such as card1 and DeviceInfo.

2.2.7 Numerical Feature Scaling

Numerical features were standardised using StandardScaler to ensure they have zero mean and unit variance. The scaler was fitted on the training set only and then applied to the validation and test sets to prevent data leakage.

This transformation is important because neural networks are sensitive to the scale of input features, and unscaled data can lead to unstable training and slower convergence. In total, 427 numerical columns were scaled after excluding the target variable and encoding categorical features.

2.3 Final Performance Metrics

In this project, the Precision-Recall Area Under Curve (PR-AUC) and F2 Score were selected as primary evaluation metrics due to the highly imbalanced nature of the fraud detection dataset.

Fraud detection datasets, including the IEEE-CIS dataset, typically contain very few fraudulent transactions compared to legitimate ones. This imbalance makes traditional metrics such as accuracy unreliable. For example, a model that predicts all transactions as non-fraudulent could still achieve very high accuracy, despite failing completely at detecting fraud.

2.3.1 PR-AUC

PR-AUC addresses this limitation by focusing specifically on the performance of the model in identifying the positive class (fraudulent transactions). It evaluates the trade-off between precision and recall.

- $Precision = \frac{TP}{TP+FP}$
- $Recall = \frac{TP}{TP+FN}$

The PR curve would plot precision against recall at different classification thresholds, and the area under this curve would summarise the overall performance.

2.3.2 F2 Score

We initially considered using the F1 score in order to select the best threshold for classifying fraud. This was the metric used for Models 1 to 3.

$$F1 = \frac{2 * Precision * Recall}{Precision + Recall}$$

However, we wanted to place more importance on the recall over precision, in order to more heavily penalise false negatives. As such, for our final model, we decided to use the F2 score instead, where we can more precisely decide the importance of recall in the metric.

3 Deep Learning Models

Section 3.4 covers the final model we had decided on for this project. Sections 3.1 to 3.3 explain the different models we had explored before arriving at our final model to document our learning process.

3.1 Model 1 - Naive Model

3.1.1 Overview

We trained a minimal feedforward neural network using PyTorch as a naive baseline for the fraud detection task. The full implementation can be found in Model_1.ipynb. The purpose of this model is to establish a lower bound on performance under the simplest possible design choices, no regularisation, no tuning, and a regression-style loss in order to motivate the need for more principled approaches in subsequent models.

3.1.2 Model Architecture

The model accepts both numerical and categorical features. Categorical variables are passed through per-column embedding layers, where each category is mapped to a dense vector of dimension min(50, (vocab_size + 1) // 2). The resulting embeddings are concatenated with the numerical features to form a single flat input vector.

This input is then passed through a fixed two-layer fully connected network:

Input → 64 → 64 → Output

Each hidden layer uses a ReLU activation with no dropout or batch normalisation. The final layer applies a sigmoid activation directly, outputting a probability in [0, 1].

Notably, unlike later models, no pos_weight adjustment is applied to account for class imbalance, and the model makes no architectural concession to the rarity of fraudulent transactions.

3.1.3 Hyperparameters and Training Setup

The model was trained using the Adam optimiser with a fixed learning rate and no learning rate scheduler. No hyperparameter search was performed; all values were set manually as a naive starting point. The training configuration is as follows:

- Optimiser: Adam
- Loss function: MSELoss (fraud probability treated as a regression target)
- Learning rate: 1e-3
- Batch size: 4096
- Epochs: 30

- Dropout: None
- Batch Normalisation: None
- Learning rate scheduler: None

The use of MSELoss is a deliberate naive choice: rather than treating fraud detection as a binary classification problem, the model is trained to regress towards the binary label directly. This is suboptimal for imbalanced classification tasks, as it does not explicitly penalise misclassification of the minority class.

3.1.4 Training Curves

The training behaviour is shown below:

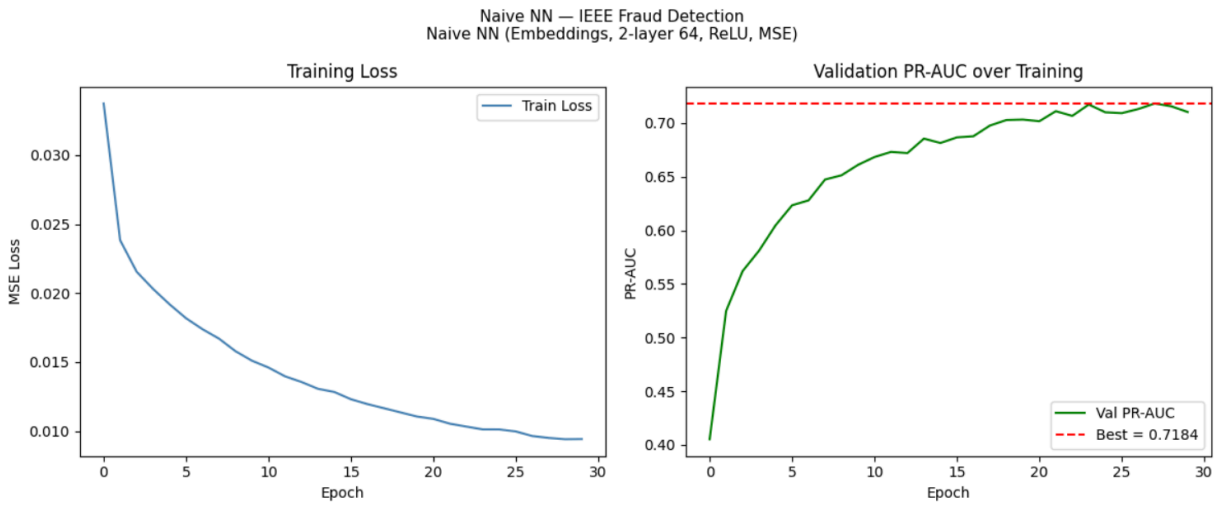

Figure 1: Neural Network Training Curves

The training loss decreases smoothly over 30 epochs, indicating that the model converges without instability.

The PR-AUC curve is shown below:

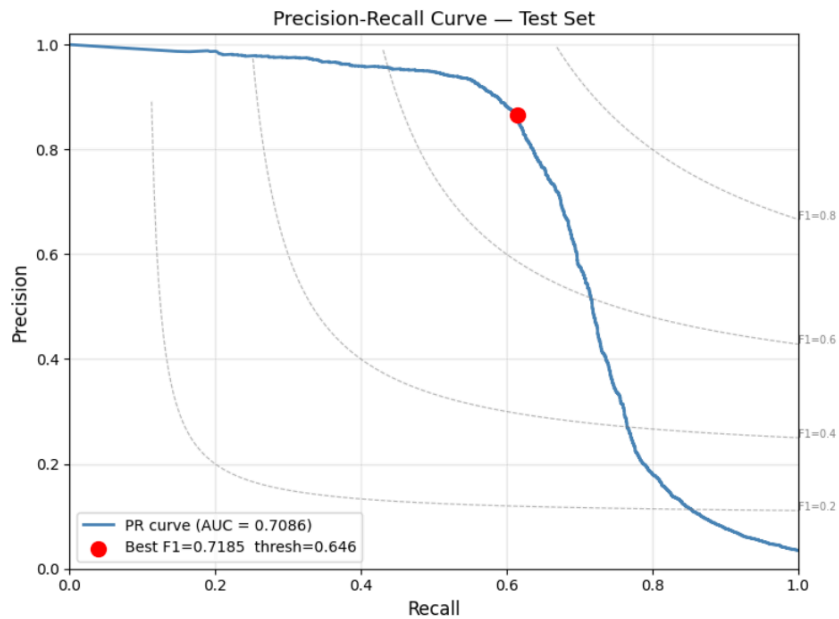

Figure 2: PR-AUC Curves

3.1.5 Results

The results are as shown below:

```
====================================================
  Checkpoint PR-AUC (recall deferred to PR curve)
====================================================
  Val  PR-AUC : 0.7184
  Test PR-AUC : 0.7086
```

Figure 3: PR-AUC Results for Model 1

```
====================================================
  Final Results at best threshold = 0.646
====================================================
  Test PR-AUC   : 0.7086
  Test F1       : 0.7185
  Test Recall   : 0.6141
  Test Precision: 0.8658
  Val  Recall   : 0.5924
```

Figure 4: Results for Model 1

```
Classification Report (threshold=0.646):
              precision    recall  f1-score   support

   Non-Fraud       0.99      1.00      0.99     85482
       Fraud       0.87      0.61      0.72      3099

    accuracy                           0.98     88581
   macro avg       0.93      0.81      0.85     88581
weighted avg       0.98      0.98      0.98     88581
```

Figure 5: Classification Report for Model 1

3.1.6 Analysis of Results

The naive model's limited performance can be attributed to several fundamental design shortcomings. The use of MSELoss treats the problem as regression rather than binary classification, which does not directly optimise for the precision-recall trade-off that is critical under severe class imbalance. The shallow two-layer architecture with only 64 units per layer lacks the representational capacity to capture the complex, non-linear feature interactions that characterise fraudulent behaviour in high-dimensional transactional data.

Furthermore, the absence of any regularisation mechanism, such as dropout or batch normalisation, combined with no pos_weight adjustment, means the model is both susceptible to overfitting and implicitly biased towards the majority (non-fraud) class. The high variance observed in the validation PR-AUC curve is consistent with this instability.

3.1.7 Conclusion

Overall, the naive model confirms that a minimal, untuned architecture with a regression-style loss is insufficient for high-performance fraud detection. The low PR-AUC and unstable validation behaviour motivate the introduction of classification-appropriate losses, improved regularisation, and more expressive architectures in subsequent models.

3.2 Model 2

3.2.1 Overview

We trained a deeper feedforward neural network using PyTorch as the second iteration of the fraud detection model. The full implementation can be found in Model_2.ipynb. This model introduces several improvements over Model 1, addressing its key shortcomings: a proper classification loss with class imbalance correction, a deeper and wider architecture, GELU activations, Batch Normalisation, dropout regularisation, gradient clipping, and a learning rate scheduler. Together, these changes represent a shift from a naive regression baseline to a more principled deep learning approach.

3.2.2 Model Architecture

Like Model 1, categorical variables are passed through per-column embedding layers using the same embedding dimension heuristic: min(50, (vocab_size + 1) // 2). The resulting embeddings are concatenated with numerical features to form the input vector.

This input is passed into a four-layer fully connected network with a progressively compressing width:

Input → 512 → 256 → 128 → 64 → Output

Each hidden layer follows the structure:

BatchNorm → Linear → GELU → Dropout

Unlike Model 1, which placed activation functions after linear layers with no normalisation or regularisation, Model 2 applies Batch Normalisation before each linear transformation to stabilise the input distribution. Dropout rates are applied per layer (0.3, 0.3, 0.2, 0.2), decreasing in the later layers to preserve more information as the representation becomes more compressed. All linear layer weights are initialised using He initialisation, appropriate for networks with ReLU activations. The final layer outputs a raw logit, with sigmoid applied during evaluation.

3.2.2 Improvements over Model 1

Compared to Model 1, this architecture introduces five key improvements:

- **Classification loss with pos_weight:** BCEWithLogitsLoss replaces MSELoss, treating fraud detection as a proper binary classification problem. A pos_weight term, computed as the ratio of negative to positive samples, is applied to penalise missed fraudulent transactions more heavily, directly addressing class imbalance.

- **Deeper and wider architecture:** Four hidden layers (512 → 256 → 128 → 64) replace the shallow two-layer (64 → 64) design, substantially increasing representational capacity to capture non-linear feature interactions.
- **GELU activations:** GELU replaces ReLU, providing a smoother gradient signal that has been shown to improve training dynamics in deeper networks.
- **Dropout regularisation:** Per-layer dropout is introduced to reduce overfitting, which was unaddressed in Model 1.
- **CosineAnnealingLR scheduler:** The learning rate is decayed smoothly from the initial value down to 1e-6 over training, helping the model converge into flatter minima in later epochs rather than oscillating around a suboptimal solution.

Additionally, gradient clipping (max norm = 1.0) is applied to prevent exploding gradients, and weight decay (1e-4) is used for L2 regularisation.

3.2.4 Hyperparameters and Training Setup

The model was trained with the following configuration. No hyperparameter search was performed; values were selected based on common best practices for deep learning on tabular data.

- **Optimiser:** Adam (betas = (0.9, 0.999), eps = 1e-8)
- **Weight decay:** 1e-4
- **Loss function:** BCEWithLogitsLoss (with pos_weight)
- **Learning rate:** 1e-3
- **Learning rate scheduler:** CosineAnnealingLR (T_max = 30, eta_min = 1e-6)
- **Batch size:** 4096
- **Epochs:** 30
- **Dropout:** 0.3, 0.3, 0.2, 0.2 (per layer)
- **Gradient clipping:** max norm = 1.0

3.2.5 Training Curves

The training behaviour is shown below:

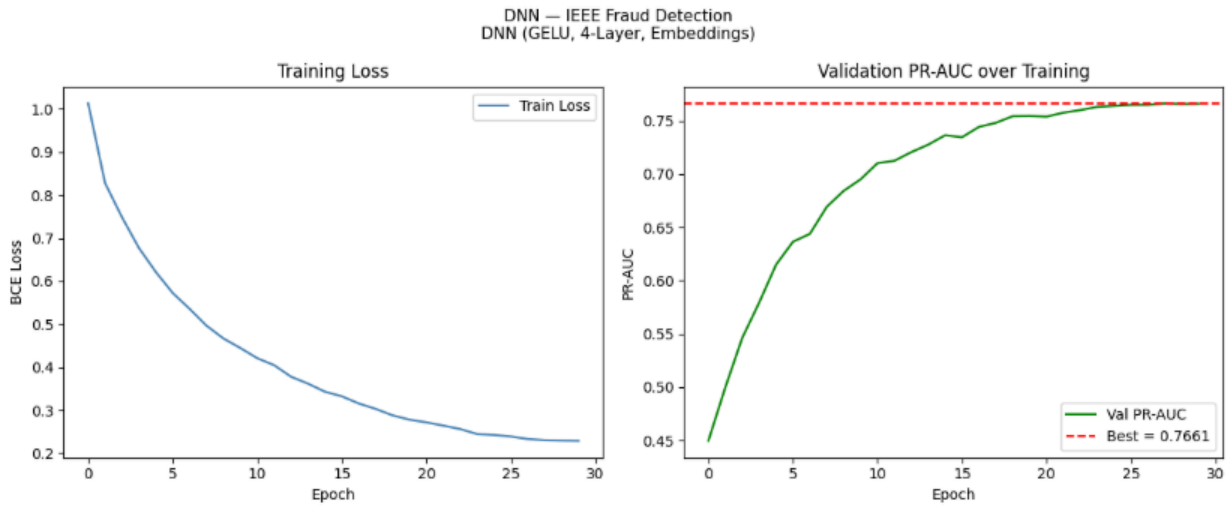

Figure 6: Neural Network Training Curves

The training loss decreases smoothly and converges across 30 epochs, reflecting stable optimisation. Validation PR-AUC improves meaningfully over Model 1, rising more rapidly in early epochs and reaching a higher plateau, demonstrating that the combination of a proper classification loss and improved architecture more effectively captures the predictive signal in the data.

The PR-AUC curve is shown below:

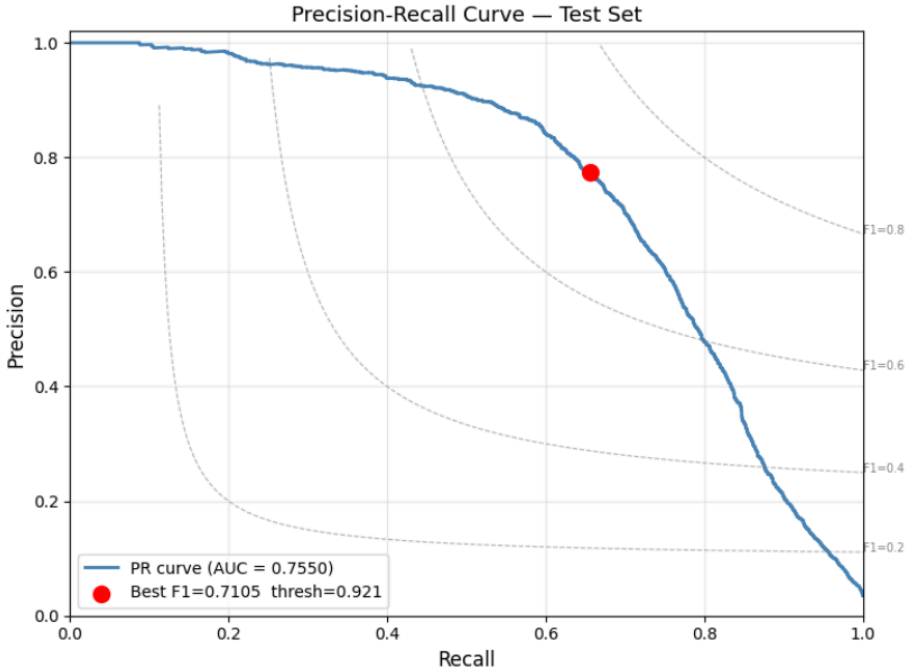

Figure 7: PR-AUC Curves

3.2.6 Results

The results are as shown below:

```
==================================================
  Checkpoint PR-AUC (recall deferred to PR curve)
==================================================
  Val  PR-AUC : 0.7661
  Test PR-AUC : 0.7550
```

Figure 8: PR-AUC Results for Model 2

```
==================================================
  Final Results at best threshold = 0.921
==================================================
  Test PR-AUC  : 0.7550
  Test F1      : 0.7105
  Test Recall  : 0.6567
  Test Precision: 0.7741
  Val  Recall  : 0.6592
```

Figure 9: Results for Model 2

```
Classification Report (threshold=0.921):
              precision    recall  f1-score   support

   Non-Fraud       0.99      0.99      0.99     85482
       Fraud       0.77      0.66      0.71      3099

    accuracy                           0.98     88581
   macro avg       0.88      0.82      0.85     88581
weighted avg       0.98      0.98      0.98     88581
```

Figure 10: Classification Report for Model 2

3.2.7 Analysis of Results

The performance gains over Model 1 can be attributed to several targeted design changes. The switch from MSELoss to BCEWithLogitsLoss with pos_weight is arguably the most impactful single change. It aligns the training objective directly with the classification task and explicitly compensates for the severe class imbalance, encouraging the model to prioritise recall on the minority fraud class. The deeper architecture with four layers allows the model to learn progressively more abstract feature interactions that the shallow two-layer design of Model 1 could not capture.

Batch Normalisation and dropout together address the training instability and overfitting tendencies observed in Model 1, while the CosineAnnealingLR scheduler provides a more principled decay of the learning rate compared to the fixed learning rate used previously.

Despite these improvements, the hidden layer widths and dropout rates in this model were manually chosen rather than empirically validated, which limits confidence that the selected configuration is close to optimal. This motivates the systematic hyperparameter search introduced in Model 3.

3.2.8 Conclusion

Overall, Model 2 represents a significant step forward from the naive baseline by introducing a proper classification objective, deeper architecture, and a suite of regularisation and training techniques. These changes result in substantially improved fraud detection performance. However, the lack of systematic hyperparameter tuning leaves performance gains on the table, motivating the structured grid search and architectural refinements in Model 3.

3.3 Model 3

3.3.1 Overview

We trained a deep feedforward neural network using PyTorch to model non-linear relationships in the dataset. The full implementation can be found in model_3.ipynb. This model builds upon Model 2 by introducing a more structured architecture, and systematic hyperparameter optimisation using grid search.

3.3.2 Model Architecture

The model consists of two main components: categorical feature embeddings and a multilayer perceptron (MLP).

Categorical variables are passed through embedding layers, where each category is mapped to a dense vector representation. This prevents the model from interpreting label-encoded categorical variables as ordinal and enables it to learn meaningful relationships between categories.

The resulting embeddings are concatenated with numerical features to form a single input vector, which is then passed into a fully connected neural network.

The architecture follows a fixed, progressively compressing structure:
Input → H1 → H2 → H3 → H4 → Output

Each hidden layer consists of:
Linear → GELU → Dropout → Batch Normalisation

The size of the first hidden layer is determined using a heuristic based on training set size and input dimensionality, while subsequent layers are reduced by a factor of two. This enforces structured dimensionality reduction throughout the network.

The model outputs a raw logit, which is converted into a probability using the sigmoid function during evaluation. To address class imbalance, we used BCEWithLogitsLoss with a pos_weight term, which increases the penalty for misclassifying fraudulent transactions and encourages higher recall on the minority class.

3.3.3 Improvements over Model 2

Compared to Model 2, this architecture introduces three key improvements:

- **Fixed structured neuron allocation:** Hidden layer sizes are deterministically defined using a scaling rule rather than being loosely selected, improving architectural consistency. This was done using the best practice formula taught in class.

    - (Number of inputs + Number of outputs) * gamma, where gamma is between 2 and 20.
- **Grid Search:** Used to systematically evaluate combinations of hyperparameters and select the best-performing configuration based on validation PR-AUC.

3.3.4 Hyperparameters and Training Setup

The model was trained using the Adam optimiser with a weight decay of 1e-5. A hyperparameter search was performed over the following space:

- γ (layer scaling factor): 4, 8, 12 (used to control hidden layer width based on the heuristic introduced in class)
- Learning rate: 1e-3, 3e-4
- Dropout: 0.2, 0.4
- Batch size: 1024, 4096

Each configuration was trained for a short number of epochs during tuning, and evaluated using validation PR-AUC. The best configuration was then retrained for the full training duration. The final training settings are as follows:

- Optimiser: Adam
- Loss function: BCEWithLogitsLoss (with pos_weight)
- Epochs: 40
- Learning rate scheduler: ReduceLROnPlateau
- Batch size: selected via tuning
- Dropout: selected via tuning
- γ (layer scaling factor): 4
- Learning rate: 0.0003
- Dropout: 0.2
- Batch size: 4096

3.3.5 Training Curves

The training behaviour is shown below:

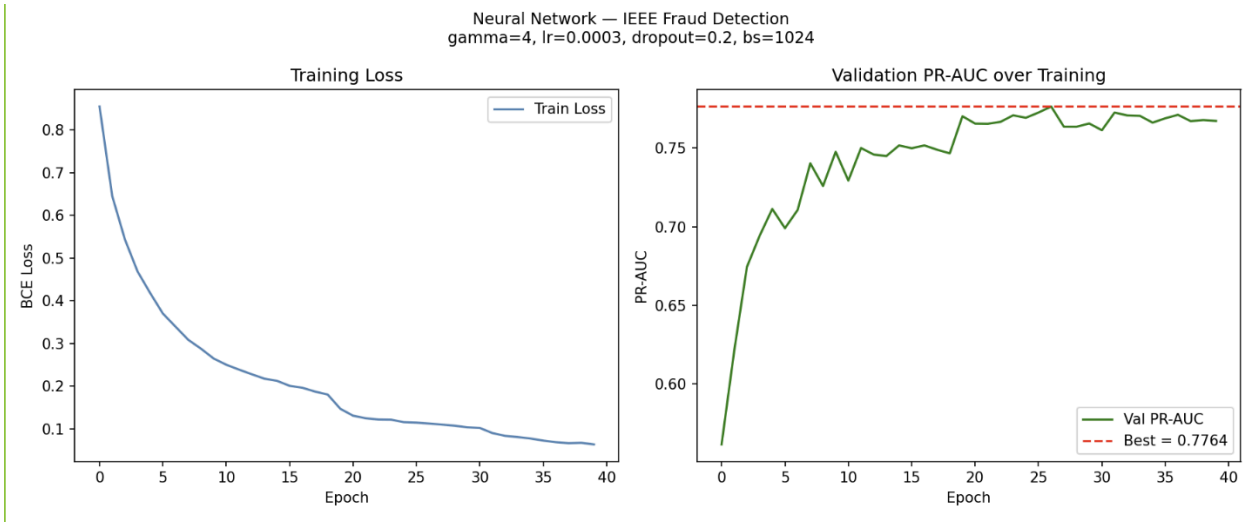

Figure 11: Neural Network Training Curves

The training loss decreases smoothly over time, indicating stable optimisation. Validation PR-AUC improves rapidly in the early epochs (till epoch 25) before plateauing, suggesting that most of the learnable signal is captured early in training.

The PR-AUC curve is shown below:

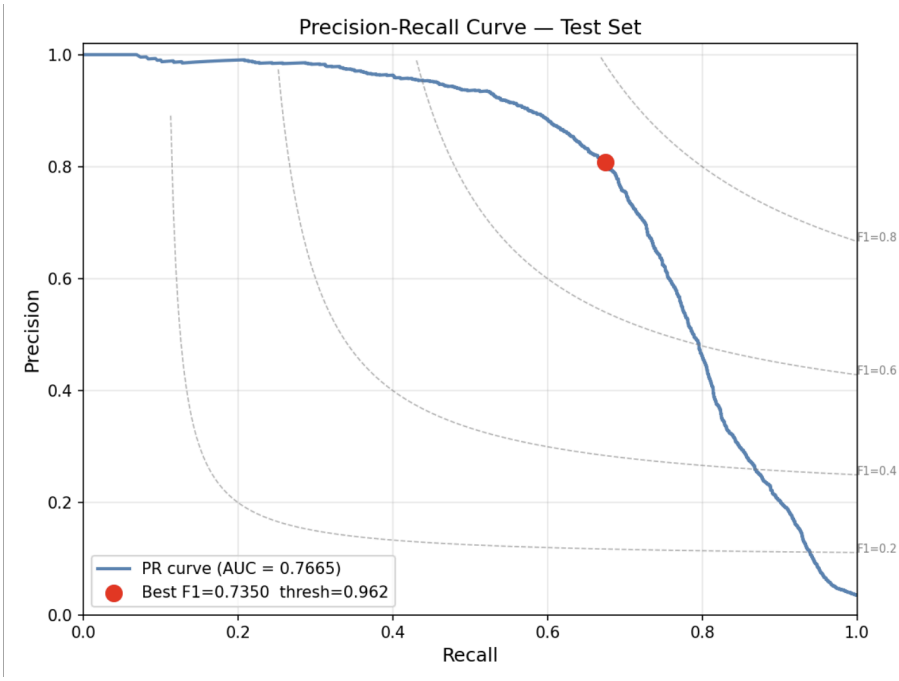

Figure 12: PR-AUC Curves

Compared to Model 2, this model achieves higher validation PR-AUC in training, indicating improved representational capacity and more efficient learning of predictive structure.

However, after reaching peak performance, further training does not yield significant gains, suggesting diminishing returns and a saturation of learnable signals under the current architecture.

3.3.6 Results

The results are as shown below:

```
==================================================
  Final Results (best checkpoint — recall deferred to PR curve)
==================================================
  Val  PR-AUC : 0.7764
  Test PR-AUC : 0.7665
```

Figure 13: PR-AUC Results for Model 3

```
====================================================
  Final Results at best threshold = 0.962
====================================================
  Test PR-AUC  : 0.7665
  Test F1      : 0.7350
  Test Recall  : 0.6744
  Test Precision: 0.8076
  Val  Recall  : 0.6805
```

Figure 14: Results for Model 3

```
Classification Report (threshold=0.962):
              precision    recall  f1-score   support

   Non-Fraud       0.99      0.99      0.99     85482
       Fraud       0.81      0.67      0.74      3099

    accuracy                           0.98     88581
   macro avg       0.90      0.83      0.86     88581
weighted avg       0.98      0.98      0.98     88581
```

Figure 15: Classification Report for Model 3

3.3.7 Analysis of Results

The improvements over Model 2 can be attributed to structured architectural design and systematic hyperparameter tuning.

The fixed layer-width schedule improves feature compression and stabilises learning by enforcing controlled dimensionality reduction.

Grid search ensures that performance gains are driven by empirically validated configurations rather than manual tuning. Despite these improvements, performance gains eventually plateau, suggesting that further improvements are more likely to come from further architectural changes.

3.3.8 Conclusion

Overall, this model improves upon Model 2 through a more structured architecture and systematic hyperparameter tuning. These changes result in improved training stability and stronger predictive performance for fraud detection.

3.4 Final model

3.4.1 Overview

We trained an enhanced deep feedforward neural network using PyTorch as the final model for fraud detection. The full implementation can be found in Final_Model.ipynb. This model builds upon Model 3 through a series of targeted improvements across every stage of the pipeline: preprocessing, model architecture, loss function, optimisation strategy, hyperparameter search, and threshold selection.

3.4.2 Pre-Processing

One key change made before training was adjusting the missing value threshold used during feature selection. In Model 3, columns with more than 75% missing values were dropped. For the final model, this threshold was raised to 90%, allowing more features to be retained. While highly sparse columns carry limited signals on average, some contain informative patterns specifically among the fraudulent minority class. Retaining these features gave the model access to additional structure that would otherwise have been discarded, contributing to improved predictive performance.

3.4.3 Model Architecture

The model retains the same foundational design as Model 3's per-column categorical embeddings concatenated with scaled numerical features, fed into a fully connected MLP, but introduces several architectural refinements.

Each categorical column was passed through its own dedicated nn.Embedding layer, with embedding dimension computed as min(50, (vocab_size + 1) // 2). The MLP follows a 3-block funnel structure compared to Model 3's 4-block design.

Input → H → H/2 → H/4 → Output

The hidden layer's structure was changed from **Linear → GELU → Dropout → BatchNorm** in model 3 to **Linear → BatchNorm → GELU → Dropout** in this Final model.

Placing BatchNorm before the activation function is the standard practice, as it normalises the pre-activation distribution and produces a more stable gradient flow.

The hidden layer widths are now determined by a tunable hidden_dim parameter (128, 256, or 512) rather than Model 3's dataset-size heuristic $H1 = n_train / (input_dim + 1) \times \gamma$. This decouples architectural width from the training set size and makes the configuration more interpretable. A floor of 32 neurons is also enforced on the final hidden layer to prevent collapse in smaller configurations.

All linear layers are initialised with Xavier uniform weights instead of He initialisation, which was used in previous models, and zero biases via an explicit _xavier_init() method.

The model outputs a raw logit, converted to a probability via sigmoid during inference.

3.4.4 Improvements over Model 3

Our final model introduced five key improvements compared to Model 3.

1. Alpha-Balanced Focal Loss:

Model 3 used BCEWithLogitsLoss with a fixed scalar pos_weight derived from the class ratio to address class imbalance. This treats every fraud sample identically, regardless of how easy or difficult it is to classify. This model replaces it with Focal Loss:

$$Loss\ function = -\alpha t\,(1 - pt)^{\gamma} log(pt)$$

The modulating factor $(1 - pt)^{\gamma}$ down-weights easy and well-classified negatives, and concentrates the gradient signal on the hard examples that the model is still getting wrong. Both α (class weight) and γ (focusing strength) are tunable hyperparameters.

2. Lookahead Optimiser with AdamW:

Model 3 uses AdamW instead of standard Adam. AdamW separates weight decay from the gradient update, allowing it to act consistently as a regularisation method and improving generalisation. On top of that, we applied a Lookahead optimiser. Instead of updating the model weights directly every step, Lookahead keeps two versions of the weights: “fast” weights that update normally, and “slow” weights that update more gradually. Every few steps, the slow weights move slightly toward the fast weights.

This approach helps to smooth out noisy updates during training, making the learning process more stable and less sensitive to fluctuations in the data. As a result, the model converges more reliably. The parameters controlling how often and how much the slow weights are updated (k and α) are tuned during training.

3. Bayesian Hyperparameter Optimisation:

Model 3 used grid search to evaluate 24 predefined hyperparameter combinations, training each configuration for 5 epochs. While straightforward, this approach is limited in scalability and does not efficiently explore larger or more complex search spaces.

In this model, hyperparameter tuning is performed using Optuna’s Bayesian optimization with the Tree-structured Parzen Estimator (TPE) sampler. Instead of exhaustively testing all combinations, the TPE sampler learns from the results of previous trials and progressively proposes more promising configurations. This allows for a more efficient search over a significantly larger 11-dimensional hyperparameter space.

A total of 30 trials are conducted, where each trial corresponds to a different hyperparameter configuration and is trained for 8 warm-up epochs. To further improve computational efficiency, unpromising trials are pruned early after 3 epochs using the MedianPruner, preventing unnecessary training on poor-performing configurations.

Additionally, several hyperparameters that were fixed in Model 3, such as weight decay, Lookahead optimiser parameters, learning rate scheduler settings, and Focal Loss parameters, are now automatically tuned. This reduces manual effort and enables the model to discover better-performing configurations.

4. Corrected LR Scheduler:

In Model 3, the learning rate scheduler monitored the metric 1 − val_pr as a workaround to convert PR-AUC into a minimisation objective. In this model, the scheduler is configured with mode = 'max', allowing it to directly track validation PR-AUC, which is more intuitive and correct.

In addition, key scheduler parameters such as patience and reduction factor are no longer fixed, but are instead tuned during the Bayesian optimisation process. This allows the scheduler behaviour to be better adapted to the training dynamics.

5. Recall-Optimised Threshold Selection:

In Model 3, the decision threshold was selected by maximising the F1 score on the test set. This introduces data leakage, as the test set should be reserved strictly for final evaluation and not used in model tuning or threshold selection.

In this model, the threshold is instead selected using the validation set, ensuring a proper separation between model development and evaluation.

Furthermore, the threshold is chosen by maximising the Fβ score with β = 2.0, which places greater emphasis on recall than precision. This is more appropriate for fraud detection tasks, where failing to identify fraudulent transactions is typically more costly than wrongly classifying transactions as fraud. This approach both aligns better with the problem objective and avoids any leakage from the test set into the model selection process.

3.4.5 Hyperparameters and Training Setup

The model was trained using AdamW wrapped in the Lookahead optimiser. A Bayesian search was performed over the following space:

Hyperparameter	Range / Options
hidden_dim	{128, 256, 512}
lr	[1e-4, 5e-3] (log-uniform)

weight_decay	[1e-6, 1e-2] (log-uniform)
batch_size	{512, 1024, 4096}
la_k (Lookahead steps)	{5, 10}
la_alpha (Lookahead interpolation rate)	[0.3, 0.8]
sched_patience	{3, 5}
sched_factor	[0.3, 0.7]
dropout	[0.1, 0.5]
alpha (FocalLoss class weight)	[0.60, 0.95]
gamma (FocalLoss focusing exponent)	[0.5, 5.0]

Each trial was trained for 8 warm-up epochs during tuning, with unpromising trials pruned by the MedianPruner after 3 epochs. The best configuration was then retrained for the full 40-epoch duration. The final training settings are as follows:

- **Optimiser:** Lookahead(AdamW)
- **Loss function:** Alpha-balanced Focal Loss (α and γ from tuning)
- **Epochs:** 40
- **Learning rate scheduler:** ReduceLROnPlateau (mode=max, tracking val PR-AUC)
- **Gradient clipping:** max_norm = 1.0
- **Threshold selection:** F-β (β = 2.0) on validation set only
- **Hyperparameter search:** Bayesian optimisation, 30 trials, TPE sampler, seeded for reproducibility
- **Best hyperparameters:** hidden dimension = 256, batch size = 1024, learning rate = 0.004477, weight decay = 0.007076, dropout = 0.4633

3.4.6 Training Curves

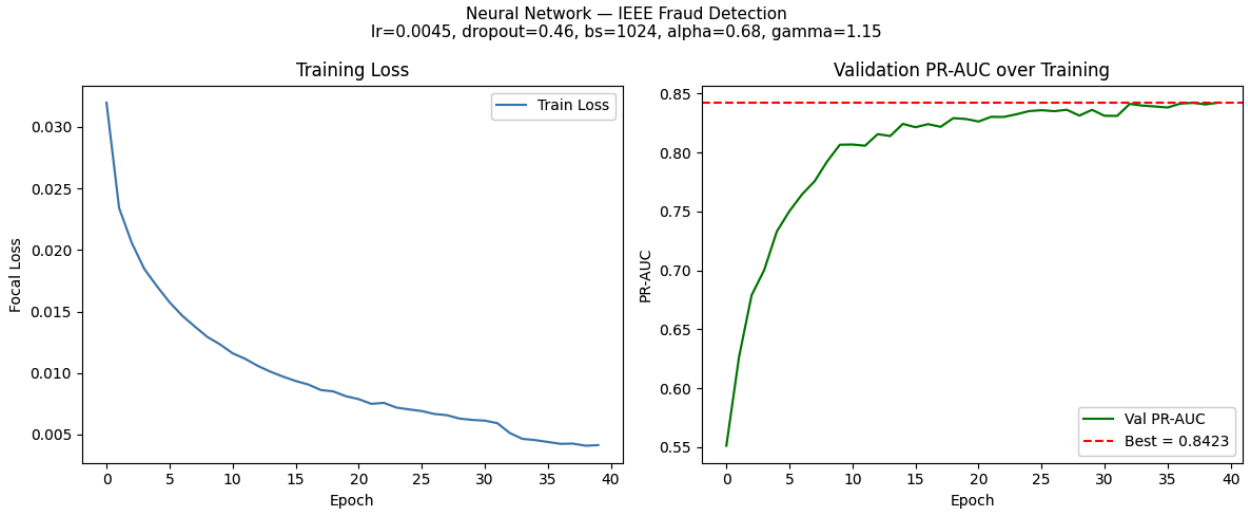

Figure 16: Neural Network Training Curves

The focal loss decreases steadily across epochs with no signs of divergence, confirming that the Lookahead–AdamW combination provides stable optimisation. Validation PR-AUC rises rapidly in the early epochs before plateauing, consistent with the pattern observed in Model 3, though at a higher peak value.

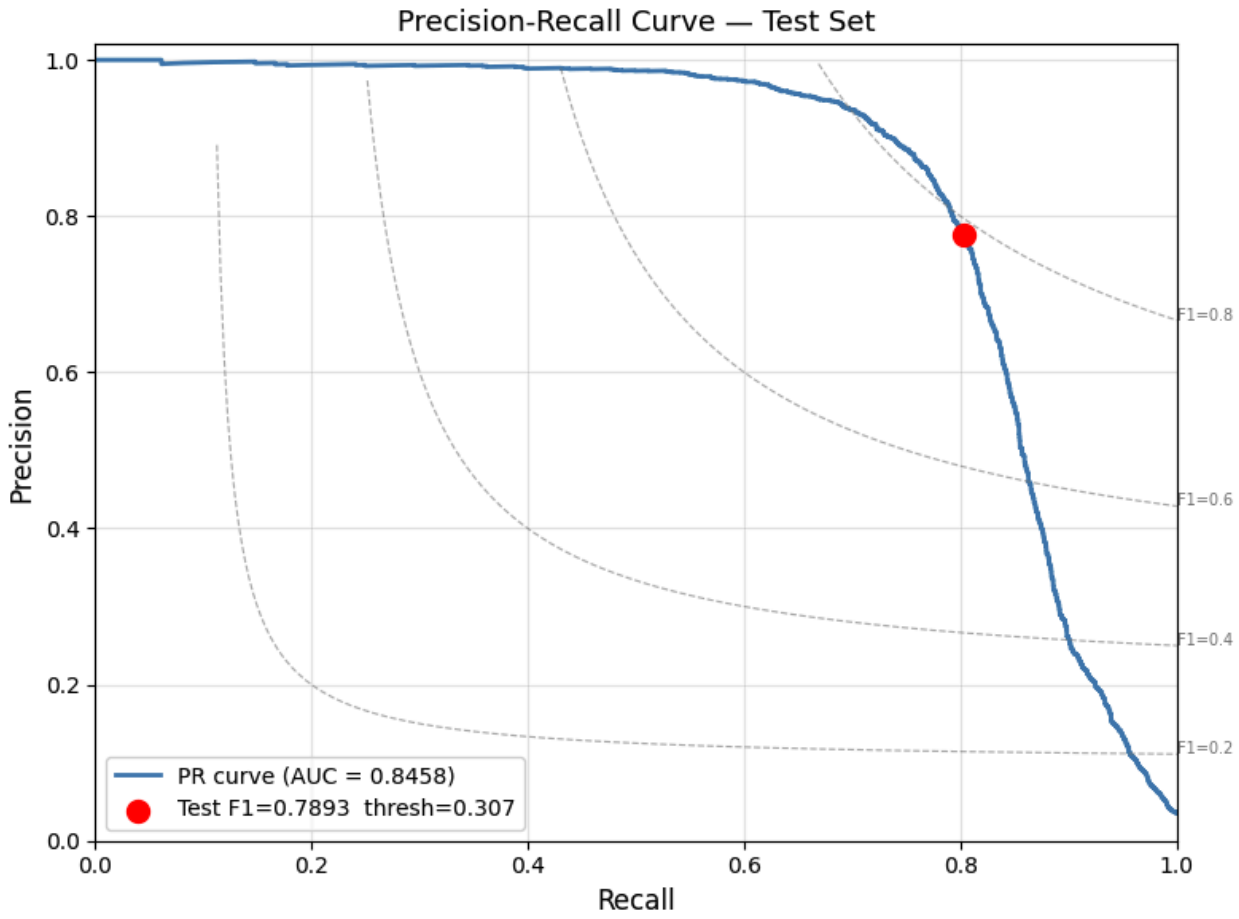

Figure 17: PR-AUC Curves

Compared to Model 3, this model achieves a higher peak validation PR-AUC, suggesting that Focal Loss and Bayesian-tuned hyperparameters together yield better discrimination of fraudulent transactions under class imbalance.

3.4.7 Results

```
==================================================
  Checkpoint PR-AUC
==================================================
  Val  PR-AUC : 0.8423
  Test PR-AUC : 0.8458
```

Figure 18: PR-AUC Results for Final Model

```
==================================================
  Final Results at validation-selected threshold = 0.307
==================================================
  Val  PR-AUC   : 0.8423
  Test PR-AUC   : 0.8458
  Val  F2.0 : 0.7983
  Val  Recall   : 0.8015
  Val  Precision: 0.7856
  Test F1       : 0.7893
  Test Recall   : 0.8032
  Test Precision: 0.7759
```

Figure 19: Results for Final model

	precision	recall	f1-score	support
Non-Fraud	0.99	0.99	0.99	85482
Fraud	0.78	0.80	0.79	3099
accuracy			0.98	88581
macro avg	0.88	0.90	0.89	88581
weighted avg	0.99	0.98	0.99	88581

Figure 20: Classification Report for Final Model

3.4.8 Analysis of Results

The improvements over Model 3 can be attributed to the compounding effect of changes made across the full pipeline, from preprocessing through to threshold selection.

Raising the missing-value threshold from 75% to 90% during preprocessing enabled the model to access additional sparse features that carry fraud-specific signals, providing richer input representations without meaningfully increasing noise for the majority class.

Focal Loss directly addresses the core limitation of BCEWithLogitsLoss with a fixed pos_weight. By applying an adaptive per-sample weighting that suppresses easy negatives and amplifies the contribution of hard, uncertain examples, it produces a more informative gradient signal, particularly critical in a highly imbalanced setting where the majority class can dominate learning.

The Lookahead and AdamW combination improves upon plain Adam by decoupling weight decay correctly to smooth out noisy parameter updates. This yields more consistent convergence and reduces the risk of settling in sharp local minima.

Bayesian optimisation explores a continuous 11-dimensional space far more efficiently than grid search, finding configurations that manual or grid-based tuning would be unlikely to discover. Crucially, it also tunes parameters such as the Focal Loss α and γ, the Lookahead step frequency, and the scheduler patience that were either fixed or absent in Model 3.

The corrected LR scheduler configuration and validation-only threshold selection via Fβ (β=2.0) further tighten the training and evaluation setup, ensuring the model is both trained and assessed in a principled, leakage-free manner that prioritises recall.

3.4.9 Conclusion

Overall, this model improves upon Model 3 through changes at every stage of the pipeline. A less aggressive feature-dropping threshold in preprocessing, a corrected and refined MLP architecture with explicit weight initialisation, a better treatment of class imbalance via Focal Loss, more stable and adaptive optimisation via Lookahead(AdamW), a far richer hyperparameter search via Bayesian optimisation, and threshold selection explicitly aligned with the fraud detection objective. Together, these changes yielded the best predictive performance for us.

4 Machine Learning Baseline Models

4.1 Overview

Baseline models were implemented to establish a reference level of performance for the fraud detection task. These models provide a point of comparison to evaluate whether more complex approaches, such as neural networks, offer meaningful improvements in predictive capability. By analysing the limitations of simpler models, we gain insight into the underlying structure of the data and the necessity of more expressive architectures.

4.2 Logistic Regression

We trained a logistic regression model using PyTorch. The code for the model can be found in Logistic_Regression.ipynb. Logistic regression provides a simple linear probabilistic classifier, serving as a baseline to assess whether the predictive signal in the dataset can be captured using linear relationships alone, and to quantify the benefit of more expressive models.

4.2.1 Model Architecture

The logistic regression model consists of a single linear layer mapping features to a logit output: Output = Linear(input_features). The model outputs a raw logit, which is transformed into a probability using the sigmoid function during evaluation. For the loss function, we chose BCEWithLogitsLoss with pos_weight for class imbalance. This increases the penalty for misclassifying fraudulent transactions, encouraging the model to prioritise recall on the minority class.

4.2.2 Hyperparameters

For our optimiser, we used Adam with a learning rate of 0.001. Our batch size was 1024. The number of epochs was 40.

4.2.3 Training Curves

The training curves are shown below:

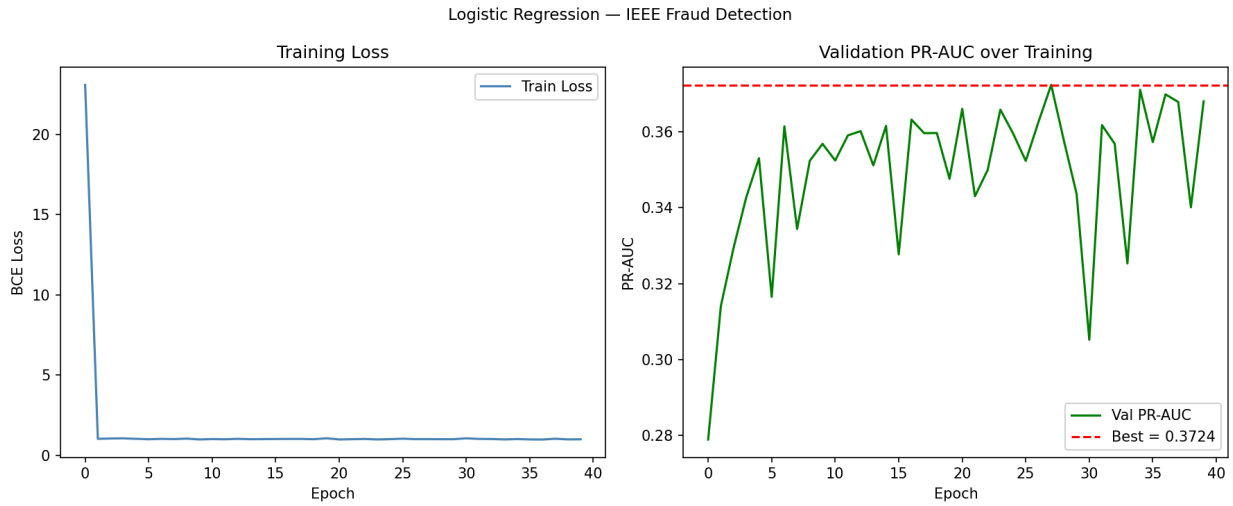

Figure 21: Training Curves

Figure 21 shows the evolution of training loss and validation PR-AUC across 40 epochs. The training loss decreases smoothly and converges, indicating stable optimisation. However, the validation PR-AUC reaches its peak early and shows minimal improvement after approximately 28 epochs. This suggests that while the model successfully minimises the loss, it is unable to learn sufficiently discriminative patterns for the fraud detection task, likely due to its limited representational capacity.

The PR-AUC curve is shown below:

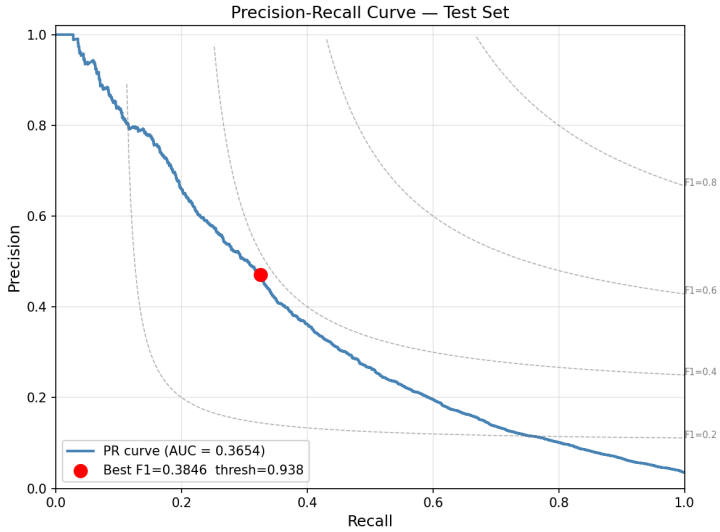

Figure 22: PR-AUC Curves

Figure 22 displays the precision-recall curve on the test set. The curve demonstrates limited but non-trivial discrimination ability across thresholds. At the optimal threshold of 0.938, the model achieves an F1-score of 0.3846. While the model performs significantly better than random guessing, the relatively low precision and recall indicate that it struggles to effectively separate fraudulent and legitimate transactions.

4.2.4 Results

The results are as shown below:

```
==================================================
  Val  PR-AUC : 0.3716
  Test PR-AUC : 0.3636
```

Figure 23: PR-AUC Results for Logistic Regression Model

```
==================================================
  Final Results at best threshold = 0.938
==================================================
  Test PR-AUC   : 0.3654
  Test F1       : 0.3846
  Test Recall   : 0.3253
  Test Precision: 0.4704
  Val  Recall   : 0.3156
```

Figure 24: Results for Logistic Regression Model

```
Classification Report (threshold=0.938):
              precision    recall  f1-score   support

   Non-Fraud       0.98      0.99      0.98     85482
       Fraud       0.47      0.33      0.38      3099

    accuracy                           0.96     88581
   macro avg       0.72      0.66      0.68     88581
weighted avg       0.96      0.96      0.96     88581
```

Figure 25: Classification Report for Logistic Regression Model

4.2.5 Analysis of Results

The logistic regression model demonstrates limited discrimination ability on the fraud detection task, achieving a test PR-AUC of 0.3654. While this performance is substantially above the random baseline, it remains significantly lower than that of more expressive models.

The recall of 0.3253 indicates that the model is able to identify a portion of fraudulent transactions, but still misses a considerable number of cases. The precision of 0.4704 suggests that a notable proportion of predicted fraud cases are false positives, reflecting limited confidence in its predictions.

The close alignment between validation and test PR-AUC (0.3724 vs. 0.3654) indicates that the model generalises consistently, and that the performance limitation arises from underfitting rather than overfitting.

This behaviour can be attributed to the fundamental limitations of logistic regression. As a linear model, it assumes that the log-odds of the target can be expressed as a linear combination of input features. However, fraud detection often depends on complex nonlinear relationships and feature interactions, which cannot be captured by a linear decision boundary.

Furthermore, the use of label-encoded categorical variables may introduce artificial ordinal relationships, which can distort the model's ability to learn meaningful patterns.

4.2.6 Conclusion

Overall, the results demonstrate that while logistic regression is able to capture some linear signal in the data, it lacks the representational capacity required for high-performance fraud detection. This justifies the use of more sophisticated models capable of learning nonlinear feature interactions.

4.3 Random Forest

We trained a Random Forest classifier using scikit-learn as the second baseline model. The full implementation can be found in Random_Forest.ipynb. Random Forest extends logistic regression by constructing an ensemble of decision trees, each trained on a bootstrapped subset of the data with random feature subsampling at each split. This allows the model to capture non-linear relationships and feature interactions that logistic regression fundamentally cannot, providing a stronger and more representative baseline for comparison with the deep learning models.

4.3.1 Model Architecture

Random Forest is a tree-based ensemble method. Each tree in the forest is grown by recursively partitioning the feature space to maximise class purity at each node, with the feature considered at each split drawn randomly from a subset of all features. Final predictions are obtained by averaging the predicted fraud probabilities across all trees.

To address class imbalance, class_weight='balanced' is applied, which automatically adjusts the per-class loss contribution inversely proportional to class frequencies. This is equivalent in effect to the pos_weight approach used in the deep learning models.

4.3.2 Hyperparameters

A baseline Random Forest was first trained with 100 trees and default settings to establish an initial performance level. Hyperparameter tuning was then performed using RandomizedSearchCV with 3-fold stratified cross-validation, optimising for ROC-AUC. The search space covered the following parameters: number of trees (100–500), maximum tree depth (unconstrained or 10–40), minimum samples per split (2–10), minimum samples per leaf (1–8), maximum features per split (sqrt, log2, 0.3, 0.5), bootstrapping (True/False), and subsampling fraction (0.6–0.8–1.0). 20 random configurations were evaluated in total, and the best-performing configuration was retrained on the full training set.

4.3.3 Training Curves

Unlike gradient-based models, Random Forest does not produce a loss curve over epochs. In place of training curves, the top 10 hyperparameter configurations from the randomised search are ranked by their cross-validation ROC-AUC, alongside a comparison of validation and test PR-AUC for the final model.

The training curves are shown below:

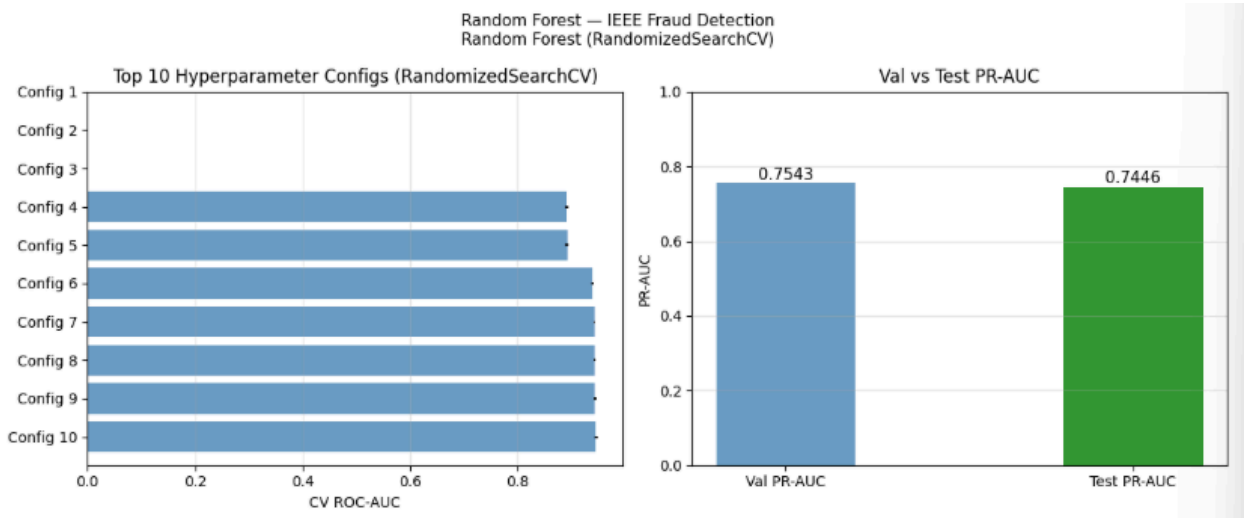

Figure 26: Training Curves

The search results show that performance is relatively consistent across the top configurations, suggesting that the model is not highly sensitive to the specific hyperparameter values within the explored range.

The PR-AUC curve is shown below:

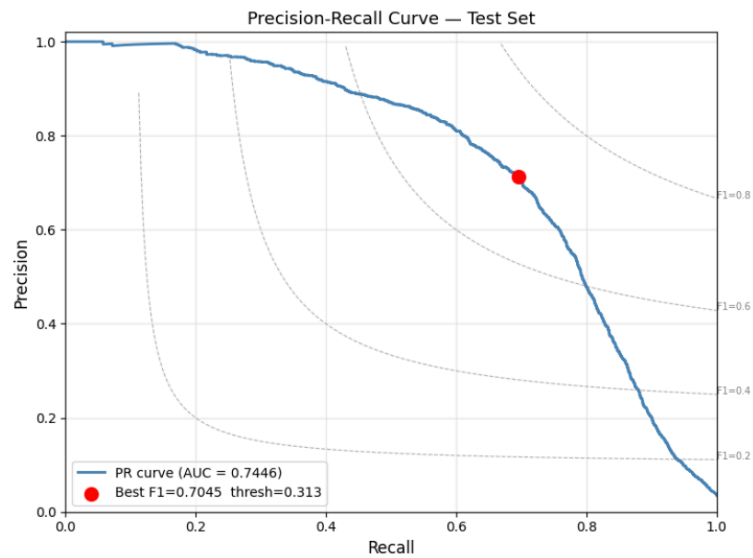

Figure 27: PR-AUC Curves

4.3.4 Results

The results are as shown below:

```
==================================================
  Checkpoint PR-AUC (recall deferred to PR curve)
==================================================
  Val  PR-AUC : 0.7543
  Test PR-AUC : 0.7446
```

Figure 28: PR-AUC Results for Random Forest Model

```
==================================================
  Final Results at best threshold = 0.313
==================================================
  Test PR-AUC   : 0.7446
  Test F1       : 0.7045
  Test Recall   : 0.6954
  Test Precision: 0.7138
  Val  Recall   : 0.6728
```

Figure 29: Results for Random Forest Model

```
Classification Report (threshold=0.313):
              precision    recall  f1-score   support

   Non-Fraud       0.99      0.99      0.99     85482
       Fraud       0.71      0.70      0.70      3099

    accuracy                           0.98     88581
   macro avg       0.85      0.84      0.85     88581
weighted avg       0.98      0.98      0.98     88581
```

Figure 30: Classification Report for Random Forest Model

4.3.5 Analysis of Results

The Random Forest model substantially outperforms logistic regression, demonstrating the importance of non-linear modelling capacity for this task. The ability to capture feature interactions and non-linear decision boundaries, which logistic regression assumes away entirely, is clearly beneficial given the complex, high-dimensional nature of the transaction data.

The close alignment between validation and test PR-AUC indicates that the model generalises well and that the randomised search did not overfit to the validation signal. The use of class_weight='balanced' effectively directs the ensemble's attention toward the minority fraud class, contributing to improved recall relative to the logistic regression baseline.

Additionally, the feature importance analysis reveals which variables contribute most to the model's predictions, providing an interpretability advantage over deep learning models. This is a useful property in fraud detection contexts where regulatory or operational requirements may demand explainability.

Despite these strengths, Random Forest has inherent limitations compared to neural network approaches. It does not natively support learned embeddings for categorical variables, relying instead on label-encoded integers, which introduce implicit ordinal assumptions. High dimensionality also poses challenges, as the model must consider all features simultaneously without the structured dimensionality reduction enforced by the MLP architectures in Models 2 and 3.

4.3.6 Conclusion

Overall, the Random Forest model confirms that non-linear modelling capacity is necessary for high-performance fraud detection, achieving meaningfully higher PR-AUC than logistic regression. However, its reliance on label-encoded categoricals and lack of learned representations limit its ceiling performance relative to the deep learning models. This motivates the continued use of embedding-based neural architectures in the main modelling pipeline.

4.4 XGBoost

An XGBoost model was trained as a second baseline to evaluate whether a gradient-boosted tree ensemble could better capture the nonlinear structure present in the fraud detection data. The code for the model can be found in xgboost.ipynb.

4.4.1 Model Architecture

The XGBoost model uses an ensemble of gradient-boosted decision trees with the binary:logistic objective, outputting a probability score directly. Each tree is grown using the histogram-based method, which enables efficient computation on large datasets. Class imbalance was addressed by setting scale_pos_weight to the ratio of negative to positive training samples (27.6), which upweights the gradient contribution of fraudulent transactions and encourages the model to maintain sensitivity to the minority class.

4.4.2 Hyperparameters

The key hyperparameters used are listed below. A maximum tree depth of 6 was chosen to allow the model to capture moderately complex interactions without excessive overfitting. The learning rate was set to 0.05, promoting gradual and stable boosting. Regularisation was applied via gamma = 1.0, reg_alpha = 0.1, and reg_lambda = 1.0 to penalise model complexity. Stochastic subsampling of both rows and columns per tree was used to reduce variance. The minimum child weight was set to 5 to prevent splits on small or noisy partitions.

Training ran for up to 1,000 boosting rounds with early stopping triggered if validation PR-AUC did not improve for 50 consecutive rounds. The best checkpoint was restored at the end of training.

4.4.3 Training Curves

The training curves are shown below:

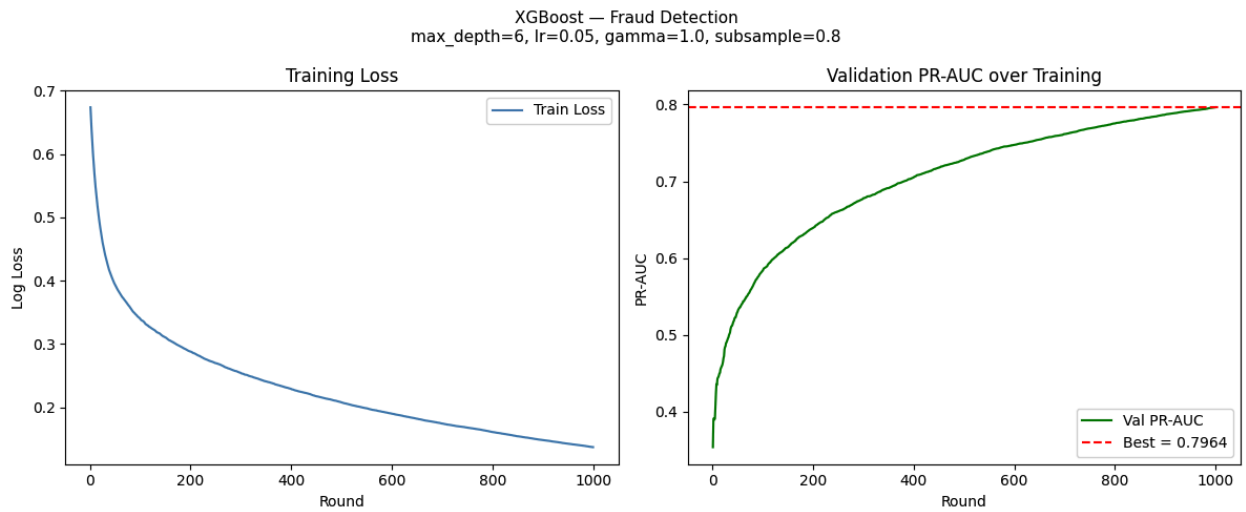

Figure 31: Training Curves

Figure 31 illustrates the evolution of training log-loss and validation PR-AUC over 1,000 boosting rounds. The training loss decreases smoothly and continuously throughout training, indicating stable gradient descent. The validation PR-AUC shows a consistent and sustained upward trend across all 1,000 rounds, with the best score of 0.7964 achieved at round 1,000. Notably, early stopping did not activate, as the model continued to improve incrementally without plateauing. This behaviour indicates that the model progressively learns more discriminative representations of fraud patterns throughout training, rather than overfitting early.

The PR-AUC curve is shown below:

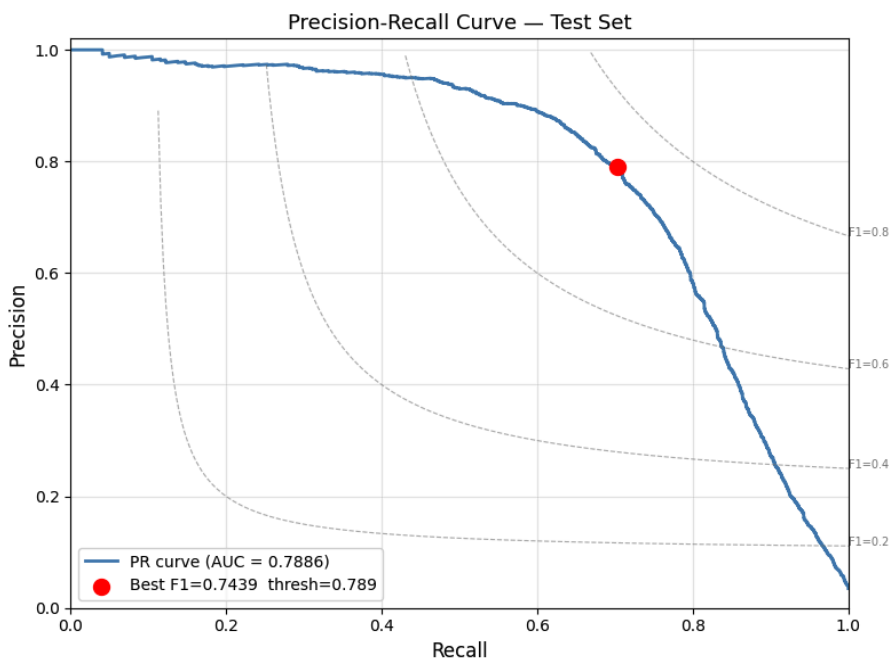

Figure 32: PR-AUC Curve

Figure 32 displays the precision-recall curve evaluated on the test set. The curve demonstrates strong discrimination across a wide range of operating thresholds. At the optimal threshold of 0.789, selected to maximise F1-score, the model achieves a precision of 0.7900 and a recall of 0.7028, yielding an F1-score of 0.7439. This represents a substantial improvement over the logistic regression baseline, reflecting the model's ability to identify nonlinear patterns in the data.

4.4.4 Results

The results are shown below:

```
==================================================
  Checkpoint PR-AUC (recall deferred to PR curve)
==================================================
  Val  PR-AUC : 0.7964
  Val  ROC-AUC: 0.9664

==================================================
  Checkpoint PR-AUC (recall deferred to PR curve)
==================================================
  Test PR-AUC : 0.7886
  Test ROC-AUC: 0.9671
```

Figure 33: PR-AUC Results for XGBoost Model

```
==================================================
  Final Results at best threshold = 0.789
==================================================
  Test PR-AUC   : 0.7886
  Test F1       : 0.7439
  Test Recall   : 0.7028
  Test Precision: 0.7900
  Val  Recall   : 0.6864
```

Figure 34: Results for XGBoost Model

```
Classification Report (threshold=0.789):
              precision    recall  f1-score   support

   Non-Fraud       0.99      0.99      0.99     85482
       Fraud       0.79      0.70      0.74      3099

    accuracy                           0.98     88581
   macro avg       0.89      0.85      0.87     88581
weighted avg       0.98      0.98      0.98     88581
```

Figure 35: Classification Report for XGBoost Model

4.4.5 Analysis of Results

The XGBoost model achieves a test PR-AUC of 0.7886 that reflects solid performance specifically on the minority class.

The recall of 0.7028 indicates that the model correctly identifies approximately 70% of all fraudulent transactions. An F1-score of 0.7439, indicating that the model handles the class imbalance effectively without collapsing into predicting only the majority class.

The close alignment between validation and test PR-AUC (0.7964 vs. 0.7886) indicates that the model generalises well to unseen data and is not overfitting to the training distribution. The small gap between these values is consistent with what would be expected given the dataset size and the regularisation applied.

Despite its strong performance, XGBoost has limitations compared to the Final Model. It relies on fixed feature representations and cannot learn hierarchical patterns the way successive MLP layers can.

4.4.6 Conclusion

Overall, the results demonstrate that XGBoost is well-suited to the fraud detection task, significantly outperforming logistic regression and random forest by capturing nonlinear feature interactions that linear models cannot represent. While its use of scale_pos_weight partially addresses class imbalance, it remains limited by fixed feature representations. This establishes a strong baseline to justify the exploration of deep learning models capable of learning richer, more complex patterns from the high-dimensional transaction data.

5 Adversarial Attack

5.1 White-Box Attack

We evaluate the robustness of the Final model using a white-box adversarial attack, where the attacker is assumed to have full access to the model architecture and trained weights. The attack method used is ITFGSM.

Rather than taking a single large gradient step, ITFGSM applies repeated small steps in the direction that decreases the model's fraud logit, clipping the total change so it never deviates from the original input by more than ε in any feature:
$x\{t + 1\} = Clip\,\varepsilon\,\{\,xt - \alpha \cdot sign(\nabla x\,J(xt,\,y))\,\}$

Only the numerical features are being changed. Categorical inputs are discrete indices consumed by embedding layers and are not suitable for continuous gradient-based modifications. The attack is therefore limited to the numerical feature subspace in the white-box setting.

The attack was run where epsilon (ε) = 0.10, step size (α) = 0.01 and the iterations = 20. The goal of the attack is to make the fraud samples labelled as a not fraud sample. The model identifies as fraudulent samples and applies a minimal perturbation to the numerical features, such that the model reclassifies it as legitimate, so the prediction flips from 1 to 0.

5.2 Vulnerabilities in our Model

The attack was applied to all test-set samples that the clean model predicted as fraud. The results are summarised below.

Fraud samples detected before the attack	2,650
Fraud Samples detected after the attack	1,602
Evasion rate	((2,650 − 1,602) / 2,650)*100 = 39.5%

When the adversarial attack was applied, it successfully fooled the model into missing fraud in 39.5% of cases. The model's ability to catch fraud had dropped from 100% to 60.5% on the attacked samples just by making small, carefully calculated tweaks to the numerical input values.

Several reasons explain why our model is vulnerable to this:

1. The model can be "reverse-engineered" by an attacker.

The numerical features passed through a neural network, where the attacker could trace backwards through the model to figure out exactly which small change to input that will cause the model to stop flagging a transaction as fraud. This is what makes gradient-based attacks possible in the first place.

2. The decision threshold is already low.

The model flags a transaction as fraud when its fraud confidence score exceeds 0.307, which was deliberately set low to maximise fraud detection. However, this also means an attacker only needs to nudge the confidence score down slightly to flip the prediction from fraud to legitimate, whereas a high threshold would require a much larger, more noticeable change.

Possible Defence Mechanisms:

One way to address this is through adversarial training, where the model is retrained with attacked examples included in the training data. This has been shown to make models more resistant to these kinds of attacks without significantly hurting normal performance.

Instructions for running the Model

Open our GitHub link: https://github.com/Sri0666/IEEE-CIS-Fraud-Detection-Project

Under the Preprocessing folder, you can find the notebook we used to preprocess our data. Under the Model folder, you can find the different notebooks we used to train our DL and ML models. The .pt files are also included for each model. Under the attack folder, you can find the notebook for the ITFGSM attack we have implemented.

To run the Final model, open the FinalModel_Playground.ipynb notebook in the Professor's Playground folder. Run all the cells in the notebook. It has already been configured to retrieve the model weights and parameters, and will output the necessary evaluation metrics for your checking on the test set.

To see other graphs, such as the training curves, or to view the other models that we have made, please refer to the respective notebooks under the Model folder. To retrain models, run the code in the Preprocessing notebook found in the Preprocessing folder, then run all the cells in the respective model notebook. Happy viewing!

Final Takeaways

This project highlighted how critical it is to design models with the problem context in mind, rather than relying solely on standard approaches. Early iterations showed that naive architectures and inappropriate loss functions fail to perform well under severe class imbalance. Through successive improvements, including better loss functions and more expressive architectures, we were able to significantly improve performance.

We also learned that feature removal should not be done purely based on intuition or exploratory observations alone. In our IEEE-CIS preprocessing pipeline, we initially removed columns with more than 75% missing values based on the EDA, in which we visualised the distributions of the columns with missing values. However, we later realised that this threshold was too aggressive and resulted in the loss of potentially informative features. A more appropriate approach would have been to validate the impact of feature removal through systematic experiments (which is what we tried for the final model, and decided that 90% was the best threshold). This highlighted the importance of making preprocessing decisions based on proper evaluation through running experiments rather than visual intuition alone, especially in high-dimensional datasets where many features can carry predictive signals.

More importantly, we learned that evaluation choices are just as important as model design. We initially used F1-score, but later switched to the F2-score to better reflect the problem requirements. In fraud detection, recall is more important than precision since missing a fraudulent transaction is more costly than flagging a legitimate one. The F2-score addresses this by giving higher weight to recall, making it a more suitable metric for our use case.

Appendix and References

1. IEEE-CIS Fraud Detection Dataset, Kaggle
   https://www.kaggle.com/competitions/ieee-fraud-detection